AWS VPC Peering: Connect Two VPCs and Prove Private Connectivity

Name: Vishwa | **Batch:** 2027 | **Region:** ap-south-1 (Mumbai) | **Completion date:** 29.10.2026 | EWB Cloud Labs, Project 01

1. Objective and summary

This project connects two isolated AWS VPCs with a VPC peering connection so that two private application servers can exchange traffic using only their private IPv4 addresses. A single public bastion host in VPC A is the only internet entry point and is used for administration through SSH ProxyJump.

The project demonstrates four things:

- Non-overlapping VPC and subnet CIDR planning.
- Public and private subnets with explicit route table associations.
- An active peering connection with routes on both sides and security rules that allow only the required inbound traffic.
- A failed test before peering, successful tests after peering, and a controlled failure with recovery.

Method. The network was built with Terraform and verified with Ansible playbooks. All resources carry the tags `Project = EWB-VPC-Peering` and `Student = Vishwa`.

2. Architecture

Component	Location	Role
Laptop	Local	Holds the private key and SSH config. Connects to the bastion on TCP 22
Internet gateway `ewb-igw-a`	Attached to VPC A	The only internet path in the project
Bastion `ewb-bastion`	VPC A / public subnet	Administration entry point, has the only public IPv4

Component	Location	Role
App-A `ewb-app-a`	VPC A / private subnet	Private workload
App-B `ewb-app-b`	VPC B / private subnet	Private workload. VPC B has no internet gateway
Peering `ewb-a-b-peer`	Between VPC A and VPC B	Private link between the two VPCs

Administration path: laptop to bastion to App-A or App-B, using SSH ProxyJump. **Project traffic:** App-A and App-B talk over private IPs through the peering connection. Peering does not give VPC B internet access through VPC A.

3. Resource inventory

Resource	Name tag	CIDR / private IP
VPC A	ewb-vpc-a	10.10.0.0/16
Public subnet A	ewb-a-public	10.10.1.0/24
Private subnet A	ewb-a-private	10.10.2.0/24
VPC B	ewb-vpc-b	10.20.0.0/16
Private subnet B	ewb-b-private	10.20.1.0/24
Internet gateway	ewb-igw-a	n/a
Route table (public A)	ewb-rt-a-public	n/a
Route table (private A)	ewb-rt-a-private	n/a
Route table (private B)	ewb-rt-b-private	n/a
Security group	ewb-sg-bastion	VPC A
Security group	ewb-sg-app-a	VPC A
Security group	ewb-sg-app-b	VPC B
Peering connection	ewb-a-b-peer	n/a
Bastion	ewb-bastion	10.10.1.10 + public IPv4

Resource	Name tag	CIDR / private IP
App-A	ewb-app-a	10.10.2.10, no public IP
App-B	ewb-app-b	10.20.1.10, no public IP
Key pair	ewb-lab	n/a

4. Address plan and subnets

VPC A uses 10.10.0.0/16 and VPC B uses 10.20.0.0/16. The two ranges do not overlap, which is a hard requirement: AWS refuses to create a peering connection between VPCs with overlapping CIDRs, and routing could not tell the two networks apart. Each /24 subnet sits inside its own VPC range. Choosing different subnet ranges would not have fixed overlapping VPC CIDRs.

All three subnets are in the same Availability Zone. Auto-assign public IPv4 is disabled on every subnet; only the bastion receives a public address, and only because it is enabled on that instance at launch.

5. Route tables

Every subnet is explicitly associated with its own route table. An internet gateway belongs to a VPC; it only becomes a path when a route table points a destination at it.

5.1 Before peering (Checkpoint 01)

Route table	Associated subnet	Routes
ewb-rt-a-public	ewb-a-public	10.10.0.0/16 local; 0.0.0.0/0 to ewb-igw-a
ewb-rt-a-private	ewb-a-private	10.10.0.0/16 local only. No internet route, no NAT gateway
ewb-rt-b-private	ewb-b-private	10.20.0.0/16 local only

5.2 After peering (Checkpoint 04)

Route table	Added route	Purpose
ewb-rt-a-private	10.20.0.0/16 to pcx	App-A to VPC B (forward path)
ewb-rt-b-private	10.10.0.0/16 to pcx	Return path from VPC B to VPC A
ewb-rt-a-public	10.20.0.0/16 to pcx	Lets the bastion open the SSH connection to App-B. The more specific /16 route is chosen over 0.0.0.0/0

6. Security groups

Inbound rules allow only what the lab needs. SSH and ICMP are never opened to 0.0.0.0/0. The private IP or CIDR goes in the Source field, never the Port field. Outbound rules are the default allow-all, and security groups are stateful, so replies to permitted traffic are allowed automatically. Network ACLs are the default allow-all.

Security group	VPC	Type / port	Source
ewb-sg-bastion	A	SSH / TCP 22	My current public IPv4 /32
ewb-sg-app-a	A	SSH / TCP 22	10.10.1.10/32 (bastion private IP)
ewb-sg-app-a	A	ICMP echo request (type 8, code 0)	10.20.1.10/32 (App-B)
ewb-sg-app-b	B	SSH / TCP 22	10.10.1.10/32 (bastion private IP)
ewb-sg-app-b	B	ICMP echo request (type 8, code 0)	10.10.2.10/32 (App-A)

7. Peering connection (Checkpoint 03)

The peering request is sent from VPC A (requester) to VPC B (accepter) in the same AWS account and Region. The request must be accepted even when both VPCs belong to the same account; with Terraform, `auto_accept = true` performs that accept step. The connection must show Active before routes can use it.

Active peering only creates the link. Traffic also needs routes on both sides and security group rules that allow it, which is why an Active peering status alone does not prove reachability.

8. SSH access with ProxyJump

All SSH commands run on my laptop. The private key `ewb-lab.pem` stays on the laptop and is never copied to any instance. `ProxyJump ewb-bastion` makes the bastion the connection path: it opens a separate SSH connection from its private IP to the destination. For this reason the App-B security group allows SSH from the bastion's private IP, not from my laptop.

Before peering, `ssh ewb-bastion` and `ssh ewb-app-a` work through VPC A's local route. `ssh ewb-app-b` works only after peering and all three peer routes are in place.

9. Test evidence

Each test below states the result it must meet.

Test 01: Baseline, before peering

- **Where:** App-A
- **Command:** `ping -c 4 10.20.1.10`
- **Expected:** no replies

The failure is expected because the route table of App-A's subnet has no route to 10.20.0.0/16 and no peering connection exists yet.

Test 02: Forward path, after peering and routes

- **Where:** App-A
- **Command:** `ping -c 4 10.20.1.10`
- **Expected:** 4 replies, 0% packet loss

Test 03: Reverse path

- **Where:** App-B
- **Command:** `ping -c 4 10.10.2.10`
- **Expected:** 4 replies, 0% packet loss

Test 04: Private SSH to App-B

- **Where:** laptop
- **Command:** `ssh ewb-app-b`
- **Expected:** a shell on App-B through the bastion

Test 05: Isolation

- **Where:** EC2 instance details for App-A and App-B
- **Expected:** no public IPv4 on either host

10. Controlled failure and recovery

To show that the security group rule really controls the traffic, I removed only the inbound ICMP echo request rule on `ewb-sg-app-b` (source 10.10.2.10/32) and left the SSH rule in place. The routes and the peering connection stayed unchanged.

Step	Action	Expected
1	Remove the App-B ICMP echo rule	Rule is gone, SSH rule still present
2	Ping from App-A: `ping -c 4 10.20.1.10`	No replies
3	Restore the exact rule	Rule is back
4	Ping from App-A again	4 replies, 0% packet loss

What this proves: the network path was intact the whole time. Only the destination's security group changed, and that alone stopped and then restored the traffic. A working route is not enough; the security group must also allow the traffic.

11. Forward path and return path

Forward path (App-A to App-B). App-A sends a ping to 10.20.1.10. The route table of ewb-a-private matches 10.20.0.0/16 and sends the packet to the peering connection. It arrives in VPC B, where the ewb-sg-app-b rule accepts ICMP echo requests from 10.10.2.10/32.

Return path (App-B to App-A). The echo reply is addressed to 10.10.2.10. The route table of ewb-b-private matches 10.10.0.0/16 and sends the reply back through the peering connection into VPC A. The security group is stateful, so the reply is allowed without a separate outbound rule.

SSH path. From the laptop, the connection goes to the bastion's public IP through the internet gateway. The bastion then opens a new SSH connection to 10.20.1.10. This traffic leaves the public subnet using the more specific 10.20.0.0/16 peering route, and the reply returns through ewb-rt-b-private.

12. Explanation questions

1. Why is the return route needed? A peering connection is only a link. Replies are new packets that need a route back to the sender's CIDR. Without the 10.10.0.0/16 route in VPC B, App-B receives the request but its reply has nowhere to go, so ping and SSH fail.

2. Why is the App-B SSH source the bastion private IP? With ProxyJump the bastion opens a second, separate SSH connection to App-B. App-B therefore sees the source address 10.10.1.10, not my laptop's public IP. The rule is a /32 so that only that one host can connect.

3. Why does Active peering not guarantee a successful ping? Active means the link exists. A ping also needs route table entries on both sides, a security group rule that allows ICMP echo on the destination, correct host IPs, and no blocking network ACL.

4. Can VPC B use VPC A's internet gateway over peering? No. Peering does not forward traffic to an internet gateway, NAT device or other gateway in the peer VPC. VPC B has no internet gateway and no internet route, and peering does not change that.

Core concepts. Peering is one-to-one and non-transitive: A-B plus A-C does not create B-C connectivity. Routes and security rules stay configurable after the connection is accepted, but changing the paired VPCs requires a new connection.

13. Cleanup (evidence)

After saving all evidence, I removed only the resources created for this project: the three instances and their volumes, the peer routes and the peering connection, the security groups, subnets and route tables, the internet gateway, both VPCs and the lab key pair. I then filtered by the tag `Project = EWB-VPC-Peering` to confirm nothing remained.

Resource types removed and checked:

- EC2 instances and their volumes
- Elastic IPs and public IPv4 allocations
- Peering connections
- Security groups, subnets, route tables and the internet gateway
- Both VPCs and the lab key pair

14. What I learned

- A peering connection, route tables and security groups are three separate controls, and traffic works only when all three agree.
- Routes must exist in both directions. A one-way route lets the request in and loses the reply.
- A more specific route (10.20.0.0/16 to the peering connection) wins over the 0.0.0.0/0 internet route, which is why the bastion subnet needs its own peering route.
- A bastion with ProxyJump lets private hosts stay without public IPs and keeps the private key on my laptop.
- Testing a failure before and during the build, and not only the success, is what shows which control is doing the work.
- Tagging every resource and cleaning up in dependency order avoids leftover cost.